Dashboard InfluxDB Query Specifications

Available Properties in InfluxDB

The following properties are collected and stored in InfluxDB:

1. **driverInputVoltage** - Driver input voltage (Vrms)
2. **driverEnergyConsumption** - Active input energy (Wh)
3. **driverInputPower** - Active input power (W)
4. **driverOperationTime** - Driver operation time (s)
5. **driverTemperature** - Driver temperature (°C)
6. **errorOverall** - Overall error flag
7. **lampOperationTime** - Lamp operation time (s)
8. **lightLevel** - DALI control gear light level (%)
9. **errorBits** - Error bits
10. **errorBitsCoupler** - Error bits coupler

InfluxDB Data Schema

Based on `services/api/src/services/mqttSubscriber.ts`, the data is stored in InfluxDB with the following structure:

Measurement

- **Name**: `dali_property`

Tags

- `controller`: Controller/device name (e.g., "Controller1")
- `category`: Device category (e.g., "sensors", "gear")
- `device_guid`: Unique device identifier
- `property`: Property name (e.g., "lightLevel", "driverInputPower")
- `unit`: Unit of measurement (e.g., "W", "°C", "%")
- `title`: Device title/name (e.g., "EVG A63", "Ballast A00")

Fields

- `value_num`: Numeric value (when value is a number)
- `value_str`: String value (when value is not a number)

Timestamp

- Server receive time (not the payload date/time)

Required Flux Queries for Dashboard

1. Get Latest Property Value for a Device

Purpose: Fetch the most recent value of a specific property for real-time display.

Flux Query:

```
from(bucket: "dali_devices")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "dali_property")
  |> filter(fn: (r) => r.device_guid == "${deviceGuid}")
  |> filter(fn: (r) => r.property == "${propertyName}")
  |> filter(fn: (r) => r._field == "value_num")
  |> last()
```

API Endpoint: ☑ Already exists at `/api/devices/:guid/history` (queries InfluxDB directly)

---

2. Get Historical Data for Trend Charts

Purpose: Fetch time-series data for a property over a specified time range.

Flux Query:

```
from(bucket: "dali_devices")
  |> range(start: -${timeRange})
  |> filter(fn: (r) => r._measurement == "dali_property")
  |> filter(fn: (r) => r.device_guid == "${deviceGuid}")
  |> filter(fn: (r) => r.property == "${propertyName}")
  |> filter(fn: (r) => r._field == "value_num")
  |> aggregateWindow(every: ${windowSize}, fn: mean, createEmpty: false)
  |> yield(name: "mean")
```

Window Sizes by Time Range:

- `1h`: `every: 1m` (1 minute intervals)
- `6h`: `every: 5m` (5 minute intervals)
- `24h`: `every: 15m` (15 minute intervals)
- `7d`: `every: 1h` (1 hour intervals)
- `30d`: `every: 6h` (6 hour intervals)

API Endpoint: ☑ Use existing `/api/devices/:guid/history?property=${property}&range=${range}` (queries InfluxDB)

---

3. Get Average Property Value Across Multiple Devices

Purpose: Calculate average light level, power, etc. across selected devices.

Flux Query:

```
from(bucket: "dali_devices")
  |> range(start: -5m)
  |> filter(fn: (r) => r._measurement == "dali_property")
  |> filter(fn: (r) => r.property == "${propertyName}")
  |> filter(fn: (r) => r.device_guid == "${deviceGuid1}" or r.device_guid ==
"${deviceGuid2}" or ...)
  |> filter(fn: (r) => r._field == "value_num")
  |> last()
  |> mean()
```

New API Endpoint Needed: ❌ `/api/devices/metrics/aggregate` (to be created - queries InfluxDB)

---

4. Get Total Energy Consumption by Device

Purpose: Sum energy consumption for bar chart comparison.

Flux Query:

```
from(bucket: "dali_devices")
  |> range(start: -${timeRange})
  |> filter(fn: (r) => r._measurement == "dali_property")
  |> filter(fn: (r) => r.property == "driverEnergyConsumption")
  |> filter(fn: (r) => r._field == "value_num")
  |> group(columns: ["device_guid", "title"])
  |> last()
  |> group()
  |> sort(columns: ["_value"], desc: true)
  |> limit(n: 10)
```

New API Endpoint Needed: ❌ `/api/devices/energy-summary` (to be created - queries InfluxDB)

---

5. Count Devices by Zone

Purpose: Get device distribution for pie chart.

Flux Query:
This is better done by querying the device list API and grouping in the frontend, as zone information is not stored in InfluxDB time-series data.

Use Existing: ☑ `/api/bmsapi/dali-devices` (queries DALI controllers) - group by zones in frontend

---

6. Count Devices with Active Errors

Purpose: Display error count in KPI card.

Flux Query:

```
from(bucket: "dali_devices")
  |> range(start: -5m)
  |> filter(fn: (r) => r._measurement == "dali_property")
  |> filter(fn: (r) => r.property == "errorOverall")
  |> filter(fn: (r) => r._field == "value_num")
  |> last()
  |> filter(fn: (r) => r._value > 0)
  |> count()
```

New API Endpoint Needed: ❌ `/api/devices/metrics/aggregate` (same endpoint, includes errorCount)

---

7. Get Latest Values for Multiple Properties (Dashboard Table)

Purpose: Fetch current values for all properties to display in data table.

Flux Query:

```
from(bucket: "dali_devices")
  |> range(start: -5m)
  |> filter(fn: (r) => r._measurement == "dali_property")
  |> filter(fn: (r) => r.device_guid == "${deviceGuid}")
  |> filter(fn: (r) =>
      r.property == "lightLevel" or
      r.property == "driverInputPower" or
      r.property == "driverEnergyConsumption" or
      r.property == "driverTemperature"
  )
  |> filter(fn: (r) => r._field == "value_num")
  |> last()
  |> pivot(rowKey:["_time"], columnKey: ["property"], valueColumn: "_value")
```

New API Endpoint Needed: ❌ `/api/devices/:guid/properties/latest` (to be created - queries InfluxDB)

---

8. Get Power Consumption Trend (Area Chart)

Purpose: Display power consumption over time with area fill.

Flux Query:

```
from(bucket: "dali_devices")
  |> range(start: -${timeRange})
  |> filter(fn: (r) => r._measurement == "dali_property")
  |> filter(fn: (r) => r.property == "driverInputPower")
  |> filter(fn: (r) => r._field == "value_num")
  |> aggregateWindow(every: ${windowSize}, fn: mean, createEmpty: false)
  |> group(columns: ["device_guid", "title"])
  |> mean(column: "_value")
  |> group()
```

Use Existing: ☑ `/api/devices/:guid/history?property=driverInputPower&range=${range}` (queries InfluxDB)

New API Endpoints to Implement

1. Aggregate Metrics Endpoint

Route: `GET /api/devices/metrics/aggregate`

Query Parameters:

- `deviceIds`: Comma-separated list of device GUIDs (optional)
- `zones`: Comma-separated list of zones (optional)
- `properties`: Comma-separated list of properties (default: lightLevel, driverInputPower, driverEnergyConsumption)

Response:

```
{
  "avgLightLevel": 78.5,
  "totalEnergy": 1250.8,
  "avgPower": 95.3,
  "deviceCount": 15,
  "errorCount": 2
}
```

Implementation (`services/api/src/routes/metrics.ts`):

```
import { FastifyInstance } from 'fastify';
import { InfluxDB } from '@influxdata/influxdb-client';
import { loadConfig } from '../config/loader.js';

export default async function metricsRoutes(fastify: FastifyInstance) {
  const config = await loadConfig();
  const queryApi = new InfluxDB({
    url: config.influx.url,
```

```
    token: config.influx.token,
  }).getQueryApi(config.influx.org);

  fastify.get('/api/devices/metrics/aggregate', async (request: any, reply) =>
{
    const { deviceIds, zones, properties =
'lightLevel,driverInputPower,driverEnergyConsumption' } = request.query;

    // Build device filter
    let deviceFilter = '';
    if (deviceIds) {
      const ids = deviceIds.split(',').map((id: string) => `r.device_guid ==
"${id}"`).join(' or ');
      deviceFilter = `|> filter(fn: (r) => ${ids})`;
    }

    const propertyList = properties.split(',');
    const results: any = {};

    // Fetch average light level
    if (propertyList.includes('lightLevel')) {
      const lightLevelQuery = `
        from(bucket: "${config.influx.bucket}")
          |> range(start: -5m)
          |> filter(fn: (r) => r._measurement == "dali_property")
          |> filter(fn: (r) => r.property == "lightLevel")
          |> filter(fn: (r) => r._field == "value_num")
          ${deviceFilter}
          |> last()
          |> mean()
      `;
      const rows = await queryApi.collectRows(lightLevelQuery);
      results.avgLightLevel = rows[0]?._value || 0;
    }

    // Fetch total energy
    if (propertyList.includes('driverEnergyConsumption')) {
      const energyQuery = `
        from(bucket: "${config.influx.bucket}")
          |> range(start: -5m)
          |> filter(fn: (r) => r._measurement == "dali_property")
          |> filter(fn: (r) => r.property == "driverEnergyConsumption")
          |> filter(fn: (r) => r._field == "value_num")
          ${deviceFilter}
          |> last()
          |> sum()
      `;
      const rows = await queryApi.collectRows(energyQuery);
      results.totalEnergy = rows[0]?._value || 0;
    }

    // Fetch error count
    const errorQuery = `
```

```
      from(bucket: "${config.influx.bucket}")
        |> range(start: -5m)
        |> filter(fn: (r) => r._measurement == "dali_property")
        |> filter(fn: (r) => r.property == "errorOverall")
        |> filter(fn: (r) => r._field == "value_num")
        ${deviceFilter}
        |> last()
        |> filter(fn: (r) => r._value > 0)
        |> count()
    `;
    const errorRows = await queryApi.collectRows(errorQuery);
    results.errorCount = errorRows[0]?._value || 0;

    return results;
  });
}
```

---

2. Energy Summary Endpoint

Route: `GET /api/devices/energy-summary`

Query Parameters:

- `range`: Time range (default: 24h)
- `limit`: Number of devices to return (default: 10)

Response:

```
[
  { "deviceGuid": "abc-123", "title": "EVG A63", "energy": 120.5, "unit": "Wh"
},
  { "deviceGuid": "def-456", "title": "Ballast A00", "energy": 95.3, "unit":
"Wh" }
]
```

Implementation:

```
fastify.get('/api/devices/energy-summary', async (request: any, reply) => {
  const { range = '24h', limit = 10 } = request.query;

  const query = `
    from(bucket: "${config.influx.bucket}")
      |> range(start: -${range})
      |> filter(fn: (r) => r._measurement == "dali_property")
      |> filter(fn: (r) => r.property == "driverEnergyConsumption")
      |> filter(fn: (r) => r._field == "value_num")
      |> group(columns: ["device_guid", "title", "unit"])
```

```
        |> last()
        |> group()
        |> sort(columns: ["_value"], desc: true)
        |> limit(n: ${limit})
    `;

    const rows = await queryApi.collectRows(query);
    return rows.map((row: any) => ({
      deviceGuid: row.device_guid,
      title: row.title,
      energy: row._value,
      unit: row.unit
    }));
  });
```

---

3. Latest Properties Endpoint

Route: `GET /api/devices/:guid/properties/latest`

Query Parameters:

- `properties`: Comma-separated list of properties (optional, defaults to available properties)

Available Properties:

- `lightLevel`, `driverInputPower`, `driverEnergyConsumption`, `driverInputVoltage`
- `driverTemperature`, `driverOperationTime`, `lampOperationTime`
- `errorOverall`, `errorBits`, `errorBitsCoupler`

Response:

```
{
  "deviceGuid": "abc-123",
  "title": "EVG A63",
  "timestamp": "2026-03-02T08:00:00Z",
  "properties": {
    "lightLevel": { "value": 78, "unit": "%" },
    "driverInputPower": { "value": 95.3, "unit": "W" },
    "driverEnergyConsumption": { "value": 1250.8, "unit": "Wh" },
    "driverTemperature": { "value": 45.2, "unit": "°C" },
    "driverInputVoltage": { "value": 230.5, "unit": "Vrms" }
  }
}
```

Implementation:

```
fastify.get('/api/devices/:guid/properties/latest', async (request: any, reply)
=> {
  const { guid } = request.params;
  const { properties =
'lightLevel,driverInputPower,driverEnergyConsumption,driverTemperature,driverIn
putVoltage' } = request.query;

  const propertyList = properties.split(',').map((p: string) => `r.property ==
"${p}"`).join(' or ');

  const query = `
    from(bucket: "${config.influx.bucket}")
      |> range(start: -5m)
      |> filter(fn: (r) => r._measurement == "dali_property")
      |> filter(fn: (r) => r.device_guid == "${guid}")
      |> filter(fn: (r) => ${propertyList})
      |> filter(fn: (r) => r._field == "value_num")
      |> last()
      |> pivot(rowKey:["_time"], columnKey: ["property"], valueColumn:
"_value")
  `;

  const rows = await queryApi.collectRows(query);
  if (rows.length === 0) {
    return reply.code(404).send({ error: 'No data found' });
  }

  const row = rows[0];
  const result: any = {
    deviceGuid: guid,
    title: row.title,
    timestamp: row._time,
    properties: {}
  };

  properties.split(',').forEach((prop: string) => {
    if (row[prop] !== undefined) {
      result.properties[prop] = {
        value: row[prop],
        unit: row.unit || ''
      };
    }
  });

  return result;
});
```

Data Fetching Strategy for Dashboard

Initial Load

1. Fetch all devices from `/api/bmsapi/dali-devices`
2. Fetch aggregate metrics from `/api/devices/metrics/aggregate`
3. Fetch energy summary from `/api/devices/energy-summary`
4. Fetch historical data for selected device/property

Real-time Updates (Polling every 5 seconds)

1. Update aggregate metrics
2. Update gauge charts with latest property values
3. Append new data points to trend charts

Filter Changes

1. Recalculate metrics based on selected devices/zones
2. Fetch new historical data for selected devices
3. Update all charts with filtered data

Query Optimization Tips

1. **Use appropriate time ranges**: Don't query more data than needed
2. **Aggregate data**: Use `aggregateWindow()` for large time ranges
3. **Limit results**: Use `limit()` to cap the number of rows returned
4. **Cache device list**: Device metadata doesn't change frequently
5. **Batch requests**: Combine multiple property queries when possible
6. **Use tags for filtering**: Tags are indexed, making queries faster

---

Example: Complete Dashboard Data Fetch

```
// Fetch all data needed for dashboard in parallel
const [devices, metrics, energySummary, lightLevelHistory] = await
Promise.all([
  fetch('/api/bmsapi/dali-devices').then(r => r.json()),
  fetch('/api/devices/metrics/aggregate?deviceIds=abc,def').then(r =>
r.json()),
  fetch('/api/devices/energy-summary?range=24h&limit=10').then(r => r.json()),
  fetch('/api/devices/abc-123/history?property=lightLevel&range=24h').then(r =>
r.json())
]);
```

InfluxDB Bucket Configuration

Based on `query.md`:

- **Bucket Name**: `dali_devices`
- **Organization**: `dali`
- **Retention Policy**: `0` (infinite retention)
- **Retention Policies**: `monthly` (for aggregated data)

Summary

The dashboard will primarily use:

1. **Existing endpoint**: `/api/devices/:guid/history` for trend charts
2. **New endpoint**: `/api/devices/metrics/aggregate` for KPI cards
3. **New endpoint**: `/api/devices/energy-summary` for bar chart
4. **Existing endpoint**: `/api/bmsapi/dali-devices` for device list and pie chart
5. **New endpoint**: `/api/devices/:guid/properties/latest` for data table

All queries use the `dali_property` measurement with appropriate filters on tags (`device_guid`, `property`, `controller`) and fields (`value_num`, `value_str`).